ML and LLM Engineering Interviews

Mechanisms, RAG, Reliability, and Project Deep Dives

Interview Preparation Notes

October 2026

How to use these notes: First explain the mechanism, then state the trade-off, then describe how you would verify your claim. The English answers are concise interview versions; personalize project answers with real evidence and never invent metrics.

Contents

1 Round 1: Theory **2**

1.1 Why does attention cost grow quadratically, and how do long-context models cope? . 2
1.2 Why can temperature 0 still give different outputs? 2
1.3 Why does fine-tuning fail to add reliable new knowledge? 2
1.4 What does the KV cache save, and what does it cost in memory? 2
1.5 Why do numbers and non-English text behave badly with BPE tokenization? 3
1.6 Why does a bigger context window not replace RAG? 3

2 Round 2: Technical Scenarios **3**

2.1 Retrieval looks right, but answers are wrong. Diagnose it. 3
2.2 A tool call times out mid-action and is retried. How do you avoid a duplicate refund? 3
2.3 Hybrid search made results worse. Why? 4
2.4 Eval scores rose, but user complaints rose too. What did the golden set miss? 4
2.5 Design multi-tenant RAG where a metadata filter bug must never leak data. 4
2.6 A retrieved document contains a hidden prompt injection. Where does defense fail? . 5

3 Round 3: Project Deep Dive **5**

3.1 Why this architecture, and what did you reject? 5
3.2 Which of your numbers would fall apart under a skeptical audit? 5
3.3 What broke in production, and why did you not catch it sooner? 5
3.4 If traffic grew 10x, what fails first? 6
3.5 What would you rebuild differently, not just add? 6

4 Answering Framework **6**

1 Round 1: Theory

1.1 Why does attention cost grow quadratically, and how do long-context models cope?

For sequence length n, full self-attention computes QK^T, producing an $n \times n$ score matrix. This gives approximately $O(n^2 d)$ arithmetic and $O(n^2)$ score storage in a naive implementation. Doubling n creates four times as many query-key pairs. FlashAttention reduces memory traffic and avoids materializing the full score matrix in high-bandwidth memory, but does not remove exact full attention's quadratic arithmetic. Sparse/local attention restricts connections; some linear-attention formulations change the computation but are not generally exactly equivalent to standard softmax attention. KV caching avoids recomputing prior K/V projections during autoregressive decoding, but does not eliminate the quadratic cost of full attention in general.

Interview answer: Full self-attention compares every query with every key, producing n^2 scores and approximately $O(n^2 d)$ computation. Long-context systems use FlashAttention, sparse or local attention, and architecture-specific efficient attention. KV caching reduces repeated work during autoregressive decoding, but does not remove the fundamental cost of full attention.

1.2 Why can temperature 0 still give different outputs?

Temperature rescales logits before softmax: $P_i = \exp(z_i/T)/\sum_j \exp(z_j/T)$. In practical APIs, zero commonly means greedy decoding. If inputs, logits, and tie-breaking are identical, greedy decoding is deterministic. Variation can still arise from changed context, model version, routing, quantization, near-tied logits, floating-point or kernel nondeterminism, batching, or tie-breaking implementation.

Interview answer: Temperature zero usually selects the highest-logit token rather than sampling. Output variation can come from near-tied logits, numerical nondeterminism, changed context, model versions, or serving configurations. I would compare exact inputs, model versions, logits, decoding settings, and backend configurations before attributing it to randomness.

1.3 Why does fine-tuning fail to add reliable new knowledge?

Fine-tuning optimizes predictions on a training distribution; it does not guarantee reliable recall of every fact across all phrasings and contexts. Limited coverage, stale or contradictory examples, interference with existing capabilities, and generalization errors can all reduce reliability. Fine-tuning is useful for behavior, format, style, and task adaptation. RAG or authoritative tools are usually better for current, source-grounded information, though they too can fail through retrieval or synthesis errors.

Interview answer: Fine-tuning improves behavior on training examples but does not guarantee robust recall of every new fact. Limited coverage, conflicting data, interference, and generalization failures can reduce reliability. I would use fine-tuning for task behavior and RAG or authoritative tools for information that must be current and verifiable.

1.4 What does the KV cache save, and what does it cost in memory?

The cache stores previously computed Keys and Values so the model need not recompute old tokens' K/V projections at every autoregressive step. Approximate memory is

$$M_{KV} = 2LBTH_{kv}d_h b,$$

where L is layers, B concurrent sequences, T cached tokens, H_{kv} KV heads, d_h head dimension, and b bytes per element. The factor 2 stores Keys and Values. For $L = 32$, $B = 1$, $T = 8192$, $H_{kv} = 8$, $d_h = 128$, and BF16/FP16 ($b = 2$), this is about 1 GiB, excluding model weights and other allocations. MQA/GQA, quantization, paged caches, and bounded context can reduce pressure.

Interview answer: The KV cache stores prior Keys and Values to avoid recomputing them during autoregressive decoding. Its memory grows linearly with layers, batch size, context length,

KV-head count, head dimension, and element size. It improves decoding efficiency but limits memory and concurrency.

1.5 Why do numbers and non-English text behave badly with BPE tokenization?

BPE learns frequent subword merges from its training distribution. A number may split into uneven chunks rather than place-value-aligned digits; underrepresented scripts and rare character sequences can fragment into more tokens. This can increase context use and cost and may make patterns harder to learn, but behavior depends on the specific tokenizer and model. Tokenization alone does not determine arithmetic ability.

Interview answer: BPE learns merges from training frequencies, so rare numerical strings and underrepresented scripts may fragment into more tokens. That can increase context and inference costs and make some patterns harder to learn. I would inspect actual tokenization and evaluate language- and number-specific tasks rather than assuming every tokenizer behaves the same way.

1.6 Why does a bigger context window not replace RAG?

A context window limits how much input a model can process; it does not discover relevant information, keep external data fresh, enforce permissions, or guarantee that evidence is used correctly. RAG retrieves relevant external documents, applies access controls, and can preserve source traceability. Large context and retrieval are complementary; irrelevant context can increase cost and distract generation.

Interview answer: A larger context window increases how much information the model can process but does not solve discovery, freshness, access control, or evidence selection. RAG retrieves relevant external information and supports traceability. I would combine retrieval with an appropriate context window rather than treating either as a complete replacement for the other.

2 Round 2: Technical Scenarios

2.1 Retrieval looks right, but answers are wrong. Diagnose it.

Trace the pipeline in stages: query understanding and retrieval; evidence sufficiency and freshness; chunking, ordering, and truncation; generation and synthesis; then evaluation correctness. Inspect exact passages and citations. Run controlled tests using a curated context. If the answer improves, context construction or distractors may be responsible; if it remains wrong, investigate interpretation, generation, or the evidence itself. Do not assume retrieval is the problem simply because the final answer is wrong.

Interview answer: I would trace retrieval, evidence sufficiency, context construction, generation, and evaluation. I would inspect the exact passages, check truncation and conflicting evidence, and run controlled tests with curated context. This separates retrieval failures from grounding and reasoning failures before I change the system.

2.2 A tool call times out mid-action and is retried. How do you avoid a duplicate refund?

A timeout means the outcome is unknown, not necessarily failed. Assign a stable idempotency key to each logical refund and reuse it across retries. The provider should deduplicate the key and return the prior result. Persist operation state, validate that a reused key has the same payload, use atomic transitions for concurrent requests, and reconcile ambiguous outcomes through provider status APIs. If the provider lacks idempotency support, a local database flag alone cannot universally prevent a duplicate external effect; reconciliation and provider-specific safeguards are needed.

```
def refund(order_id, amount, idempotency_key, provider):
    # Reuse the same key for retries of the same logical refund.
    return provider.create_refund(
```

```
        order_id=order_id,
        amount=amount,
        idempotency_key=idempotency_key,
    )
```

Interview answer: I would assign a stable idempotency key to each logical refund and reuse it across retries. The provider should deduplicate requests and return the existing result. I would persist operation state, handle concurrency atomically, and reconcile ambiguous timeouts before creating any new logical refund.

2.3 Hybrid search made results worse. Why?

Hybrid retrieval combines lexical methods such as BM25 with dense vector search. It can regress because raw scores are on incompatible scales, fusion weights are poor, embeddings return semantically related but incorrect passages, exact identifiers lose rank, duplicates crowd out diverse evidence, or chunking/metadata is faulty. Compare lexical-only, dense-only, and hybrid on the same judged queries. Inspect Recall@k, MRR, nDCG@k, candidate overlap, and query categories. Reciprocal Rank Fusion (RRF) is a useful baseline:

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{k_0 + \mathrm{rank}_r(d)}.$$

Interview answer: I would compare lexical-only, dense-only, and hybrid retrieval on the same judged query set. I would inspect candidate overlap, ranking metrics, chunk quality, and failure categories, then tune rank fusion and candidate depth through ablations rather than assuming hybrid search must improve relevance.

2.4 Eval scores rose, but user complaints rose too. What did the golden set miss?

The set may overrepresent easy or common prompts and omit long-tail, multilingual, multi-turn, adversarial, abstention, privacy, or real-world task-completion cases. The metric may reward wording similarity rather than factual correctness; an LLM judge may also disagree with human judgments. Classify complaints, add representative cases and hard negatives, validate judges against human review, version the set, and track online task success, latency, safety, and user outcomes as well as offline scores.

Interview answer: I would test whether the golden set covers production traffic, difficult edge cases, multi-turn interactions, abstention, and safety failures. I would classify complaints, add representative cases, validate automated judges against human assessments, and track task success and user impact alongside offline quality metrics.

2.5 Design multi-tenant RAG where a metadata filter bug must never leak data.

Treat tenant isolation as a security invariant. Derive tenant identity from trusted authentication, not a user-controlled query parameter. Enforce tenant restrictions server-side and, where feasible, independently through row-level security, namespaces, or separate indexes. Fail closed on missing identity or filter validation failure. Verify returned document ownership, include tenant and authorization context in cache keys, audit access, and test cross-tenant negative cases. Application-level filters alone are insufficient if the same bug can omit them.

Interview answer: I would treat tenant isolation as a security invariant, not a retrieval-quality feature. Tenant identity would come from trusted authentication, and restrictions would be enforced server-side and independently where possible. I would fail closed, validate returned documents, isolate caches, and continuously test cross-tenant access paths.

2.6 A retrieved document contains a hidden prompt injection. Where does defense fail?

Retrieved content is untrusted data, not an instruction source. Delimit it clearly and preserve instruction hierarchy, but do not rely on prompt wording alone. Use least-privilege tool permissions, server-side authorization, schema and policy checks for actions, data minimization, and human confirmation for high-impact actions where appropriate. Test direct and indirect injection, hidden/encoded text, and cross-document attacks. Detection and filtering help but cannot be the only security boundary.

Interview answer: I would treat retrieved documents as untrusted data and separate instructions from evidence. I would combine prompt-level defenses with least-privilege tool access, server-side authorization, action validation, and adversarial testing. I would not rely on a prompt instruction or injection classifier as the sole security boundary.

3 Round 3: Project Deep Dive

3.1 Why this architecture, and what did you reject?

Start with requirements and measurable constraints: workload, latency, reliability, data, security, and cost. Compare a small set of plausible alternatives, explain trade-offs, and support the decision with benchmarks, prototypes, or operational evidence. State which changed assumption would make you revisit the choice. For a RAG system, possible options include managed vector search, self-hosted search, or database-native retrieval; the right answer depends on workload and constraints.

Interview answer: I selected the architecture by translating requirements into measurable constraints, comparing viable alternatives, and validating trade-offs through benchmarks or prototypes. I rejected alternatives for specific reasons such as latency, cost, complexity, or reliability, and would revisit the decision if assumptions changed.

3.2 Which of your numbers would fall apart under a skeptical audit?

Audit dataset inclusion rules, split methodology, leakage risks, baseline, metric definition, threshold, confidence intervals, subgroup performance, and production-versus-offline differences. For systems, define p50/p95/p99 latency measurement conditions and cost per request, including retries and infrastructure. Reproduce results and distinguish measured facts from estimates. Never fabricate a number to fill a gap.

Interview answer: I would scrutinize metrics whose measurement conditions, sample sizes, baselines, or assumptions are unclear. I would reproduce the result, audit the data split, quantify uncertainty, and distinguish offline performance from production outcomes. I would label estimates and limitations rather than presenting unsupported precision.

3.3 What broke in production, and why did you not catch it sooner?

Explain user impact, technical root cause, detection gap, and corrective actions. Example: updated documents but stale embeddings/index; latency and availability look healthy while answers become outdated. The missing control might be a freshness/version consistency check. Add targeted alerts, end-to-end correctness tests, canaries, rollback, incident runbooks, and regression tests. Focus on prevention rather than blame.

Interview answer: I would explain the incident through user impact, technical root cause, detection gap, and corrective actions. I would identify why existing tests or monitoring missed it, then add targeted safeguards and regression tests. The goal is to demonstrate learning and prevention, not shift blame.

3.4 If traffic grew 10x, what fails first?

Do not guess: profile the current system and load-test a realistic workload. Potential bottlenecks include GPU throughput and KV-cache memory, retrieval QPS and filtering, downstream API limits, database pools, network, and application concurrency. Measure queue depth, throughput, p95/p99 latency, errors, memory, and cost. Add batching, caching, backpressure, rate limits, autoscaling, or graceful degradation based on measured bottlenecks. A 10x request rate may imply roughly 10x token throughput only if request mix stays comparable; actual capacity depends on batching and utilization.

Interview answer: I would profile the system, identify the first saturated resource, and model capacity under a realistic 10x workload. I would load-test throughput, queueing, tail latency, memory, and cost, then address the measured bottleneck. I would also test graceful degradation and downstream rate limits.

3.5 What would you rebuild differently, not just add?

Identify recurring architectural causes of reliability, debugging, or scaling problems. Potential redesigns include reproducible pipelines instead of ad hoc scripts, versioned schemas and configuration, explicit authorization boundaries, offline plus online evaluation, staged deployment, rollback, idempotent actions, and bounded retries. Tie each change to a real failure and discuss migration cost and trade-offs rather than listing fashionable technologies.

Interview answer: I would redesign components that caused recurring reliability, debugging, or scaling problems instead of merely adding more components. I would prioritize data contracts, observability, automated evaluation, and safe deployment controls where evidence shows they would prevent failures, then validate the redesign before migration.

4 Answering Framework

For each scenario, practise this sequence:
1. Clarify objective, constraints, and what failure means.
2. State the simplest baseline or expected behavior.
3. Rank hypotheses by likelihood and impact.
4. Name the evidence or experiment that distinguishes each hypothesis.
5. Explain the fix and its trade-offs.
6. Describe validation, monitoring, rollback, and business impact.

Interview rule: Do not claim you measured something if you did not. Say what is known, what is inferred, and what you would test next.